

Relatório do Laboratório - Threads em Java

Aluno: Artur Bomtempo Colen

Professor: Cristiano de Macedo Neto

Curso: Engenharia de Software - PUC Minas

Introdução

Este relatório apresenta os resultados práticos e as respostas teóricas do roteiro de laboratório sobre Threads em Java. O objetivo do laboratório foi revisar a diferença entre processo e thread do ponto de vista do sistema operacional, implementar concorrência em Java pelas formas clássicas (`extends Thread` e `implements Runnable`) e pelas formas atuais (`ExecutorService` e `Virtual Threads`), além de observar na prática o custo de criar milhares de threads nativas do sistema operacional. O código de todas as partes foi implementado e executado com sucesso, e as respostas abaixo se baseiam tanto na teoria revisada em aula quanto nos resultados observados ao rodar cada experimento.

Processo x Thread

Um processo é a unidade que o sistema operacional cria para cada programa em execução, no nosso caso a própria JVM. Ele possui espaço de endereçamento de memória próprio, o que significa que os dados de um processo não são visíveis para outro processo diretamente. Já uma thread é uma linha de execução dentro de um processo, e todas as threads de um mesmo processo compartilham a mesma memória. Essa diferença tem consequências práticas importantes. Criar um processo novo é caro, pois o sistema operacional precisa montar uma nova tabela de páginas de memória e atribuir um novo identificador de processo (PID). Criar uma thread é bem mais leve, já que ela reaproveita a memória do processo que já existe. Em termos de comunicação, processos diferentes precisam de mecanismos como pipes ou sockets para trocar informações, enquanto threads do mesmo processo podem simplesmente compartilhar variáveis. Em termos de isolamento, uma falha grave em um processo não derruba outros processos do sistema, mas uma exceção não tratada em uma thread pode comprometer o processo inteiro, já que todas as threads dividem o mesmo espaço de memória. Por fim, quem o sistema operacional efetivamente escalona na CPU são as threads, não os processos: o processo é apenas a unidade que agrupa recursos.

Parte A: `extends Thread`

Nesta parte, a classe `AtendimentoThread` estende diretamente a classe `Thread` e sobrescreve o método `run()`. Foram criadas 5 instâncias, cada uma representando o atendimento de um cliente, todas iniciadas com `start()` e aguardadas com `join()`.

Pergunta do roteiro: o tempo total ficou perto de 1s ou de 5s com 5 atendimentos? Por quê?

Nos testes realizados, o tempo total ficou próximo de 1 segundo (aproximadamente 1013ms), e não de 5 segundos. Isso acontece porque cada chamada a `start()` cria uma thread de sistema operacional independente, e todas as 5 threads passam a rodar de forma concorrente entre si. Como cada uma delas executa `Thread.sleep(1000)` ao mesmo tempo que as outras, o tempo total do programa é limitado pela thread mais lenta, e não pela soma do tempo de todas. Se o programa tivesse chamado `run()` diretamente em vez de `start()`, ou se tivesse chamado `join()` logo após iniciar cada thread dentro do mesmo laço, os atendimentos aconteceriam um de cada vez, de forma sequencial, e aí sim o tempo total ficaria perto de 5 segundos.

Parte B: implements Runnable

Nesta parte, a classe `AtendimentoRunnable` implementa a interface `Runnable` em vez de estender `Thread`. A tarefa (o que fazer) fica separada de quem executa essa tarefa (a `Thread`), e cada `Runnable` é passado para uma nova `Thread` no momento da criação, já recebendo um nome customizado.

Pergunta do roteiro: qual das duas classes, da Parte A ou da Parte B, você poderia fazer herdar de outra classe hoje?

A resposta é a classe da Parte B, `AtendimentoRunnable`. Como ela apenas implementa a interface `Runnable`, ainda está livre para estender qualquer outra classe, por exemplo uma classe `Funcionario`, sem perder a capacidade de rodar em uma thread. Já a classe da Parte A, `AtendimentoThread`, já usou sua única herança disponível em Java ao estender `Thread`, então não teria como herdar também de `Funcionario`. Esse é justamente o motivo pelo qual, desde o Java 5, implementar `Runnable` é considerada a prática recomendada em vez de estender `Thread` diretamente.

Parte C: muitas threads de sistema operacional

Nesta parte, o programa cria 10 mil threads nativas de sistema operacional, cada uma apenas dormindo por 1 segundo, para sentir na prática o custo de memória desse modelo. O roteiro indica que, em muitas configurações, esse número acima de aproximadamente 50 mil threads já é suficiente para provocar um `OutOfMemoryError`.

Primeira pergunta do roteiro: por que criar uma thread de sistema operacional é mais caro do que criar um objeto comum em Java?

Um objeto comum em Java existe inteiramente dentro da memória gerenciada pela JVM, ocupa poucos bytes na heap e é limpo automaticamente pelo garbage collector, sem qualquer interação com o sistema operacional. Já uma thread de sistema operacional exige que o próprio SO reserve recursos reais para ela, entre eles uma pilha de execução própria, que costuma ocupar entre 512KB e 1MB, além de estruturas de controle no kernel, como registradores e o identificador da thread. Criar uma thread envolve uma chamada de sistema, o que é uma operação bem mais custosa do que simplesmente alocar memória dentro da JVM.

Segunda pergunta do roteiro: o que esse limite sugere sobre usar 1 thread por requisição em um servidor web?

Sugere que esse modelo não escala para cenários com muitos usuários simultâneos. Um servidor que atende, por exemplo, dezenas de milhares de conexões ao mesmo tempo, criando uma thread de sistema operacional por requisição, corre o risco real de esgotar a memória disponível e lançar `OutOfMemoryError`, mesmo que cada requisição individualmente não exija muito processamento. Foi exatamente esse problema de escala que motivou a evolução do modelo de concorrência em Java, primeiro para o `ExecutorService`, que limita o número de threads reais criadas, e mais recentemente para as `Virtual Threads`.

Parte D: `ExecutorService`

Nesta parte, em vez de criar uma thread nova para cada tarefa, um pool fixo de 4 threads, criado com `Executors.newFixedThreadPool(4)`, foi usado para atender 10 clientes. As tarefas são enviadas ao pool com `submit()`, e quem decide quando e qual thread vai executar cada uma é o próprio pool.

Pergunta do roteiro: com 4 threads atendendo 10 clientes, o tempo total ficou perto de 1s, 2s ou 3s?

Nos testes realizados, o tempo total ficou próximo de 3 segundos (aproximadamente 3026ms). Isso acontece porque, com apenas 4 threads reais disponíveis, somente 4 tarefas podem rodar ao mesmo tempo. As 10 tarefas acabam sendo processadas em 3 levadas, sendo 4, depois mais 4, e por fim as 2 restantes, e como cada leva demora cerca de 1 segundo, o tempo total fica perto de 3 segundos. Isso também explica por que nomes de thread como `pool-1-thread-1` se repetem entre diferentes clientes atendidos: são as mesmas 4 threads sendo reaproveitadas para tarefas diferentes.

Parte E: `Virtual Threads`

Nesta parte, o mesmo experimento de escala da Parte C foi refeito, mas usando `Executors.newVirtualThreadPerTaskExecutor()`, criando uma `Virtual Thread` para cada uma das 100 mil tarefas, sem que o programa apresente erro de memória.

Pergunta do roteiro: uma `Virtual Thread` é uma thread de sistema operacional? Se não, o que ela é?

Não, uma `Virtual Thread` não é uma thread de sistema operacional. Ela é uma thread gerenciada inteiramente pela JVM, sem correspondência direta de um para um com uma thread nativa do sistema operacional. Por baixo, a JVM mantém um número pequeno de threads reais, chamadas `carrier threads`, e vai alternando qual `Virtual Thread` está sendo executada por cada `carrier thread`, principalmente nos momentos em que uma `Virtual Thread` fica bloqueada, por exemplo durante um `Thread.sleep()` ou uma operação de entrada e saída. É justamente por isso que é possível criar centenas de milhares de `Virtual Threads` sem esgotar a memória do processo, algo que não seria viável com threads de sistema operacional tradicionais, como ficou evidente na comparação com a Parte C.

Exercícios de fixação

1) Trocar o pool fixo por `newCachedThreadPool()`. O comportamento muda de forma significativa. Com o pool fixo, `newFixedThreadPool(4)`, o tempo total ficou em torno de 3 segundos, e apenas 4 nomes de thread distintos apareceram na saída, reaproveitados entre as 10 tarefas. Já com o `newCachedThreadPool()`, o tempo caiu para cerca de 1 segundo, e apareceram 10 nomes de thread distintos, um para cada tarefa. Isso acontece porque o pool cached não tem um limite fixo de threads: ele cria uma thread nova sempre que não há nenhuma disponível no momento, e como as 10 tarefas chegam praticamente ao mesmo tempo, ele acaba criando 10 threads de uma só vez. Esse comportamento é vantajoso para picos de tarefas curtas, mas pode ser arriscado em cenários de carga alta e sustentada, já que não existe um teto para a quantidade de threads criadas.

2) Imprimir `Thread.currentThread()` na Parte E. Ao imprimir `Thread.currentThread()` dentro da tarefa executada pelo `newVirtualThreadPerTaskExecutor()`, a saída mostra algo no formato `VirtualThread` seguido de um identificador, por exemplo `VirtualThread[#27]/runnable@ForkJoinPool-1-worker-1`, um formato bem diferente do `Thread[#14,pool-1-thread-1]` visto nas partes anteriores. Além disso, chamando o método `isVirtual()`, disponível na classe `Thread` desde o Java 21, o retorno foi `true` em todos os testes, confirmando de forma definitiva que se trata mesmo de uma Virtual Thread. É possível notar ainda, no nome impresso, a identificação da carrier thread que está executando aquela Virtual Thread naquele instante.

3) Explicar processo x thread com a analogia do guichê de atendimento. Um processo pode ser comparado a uma agência de atendimento inteira: ela tem um endereço próprio, um espaço físico e seus próprios arquivos e cofres, que representam a memória do processo. Se uma agência fecha as portas por algum problema grave, as outras agências da cidade continuam funcionando normalmente, isoladas desse problema. As threads, por sua vez, são os atendentes que trabalham dentro dessa agência: todos compartilham o mesmo espaço físico, os mesmos arquivos e o mesmo cofre, o que permite que troquem informações rapidamente entre si, sem precisar de um sistema de correspondência formal como aconteceria entre agências diferentes, que seria o equivalente à comunicação entre processos. Abrir uma agência nova é caro e demorado, assim como criar um processo novo exige montar uma nova tabela de páginas de memória e um novo identificador no sistema operacional, enquanto contratar mais um atendente para uma agência que já existe é bem mais rápido e barato, da mesma forma que criar uma thread. Assim como um gerente decide qual atendente vai atender qual cliente e quando cada um tem sua vez no guichê, é o escalonador do sistema operacional quem decide quando cada thread efetivamente executa na CPU. Por fim, se um atendente comete um erro grave durante o atendimento, isso pode comprometer o funcionamento de toda a agência, assim como uma exceção não tratada em uma thread pode afetar o processo inteiro.

4) Escolher, com justificativa, a abordagem ideal para um servidor com milhares de conexões. Para um servidor com milhares de conexões simultâneas, a abordagem mais adequada hoje são as Virtual Threads, vistas na Parte E. O modelo das Partes A e B, que cria uma thread de sistema operacional por conexão, esbarra nos limites de memória observados na Parte C, algo que se torna inviável na casa de dezenas de milhares de conexões simultâneas. O ExecutorService com pool fixo, visto na Parte D, resolve o problema de memória ao limitar o número de threads reais criadas, mas troca esse problema por outro: com milhares de conexões chegando ao mesmo tempo e poucas threads fixas disponíveis para atendê-las, as requisições excedentes ficam enfileiradas esperando, o que aumenta a latência percebida pelo usuário. As Virtual Threads resolvem os dois problemas ao mesmo tempo, permitindo um modelo de programação simples, com uma thread por conexão, como nas Partes A e B, mas sem o custo de memória de uma thread de sistema operacional real, já que são gerenciadas pela JVM e mapeadas dinamicamente sobre um pequeno grupo de carrier threads, como demonstrado na Parte E.

Conclusão

O laboratório permitiu comparar, na prática, quatro formas diferentes de trabalhar com concorrência em Java, desde a mais antiga, extends Thread, até a mais recente, Virtual Threads. Ficou evidente que cada geração dessas abordagens resolveu um problema de escala específico da anterior, sem descartar os conceitos fundamentais de sistema operacional revisados no início do roteiro, como a diferença entre processo e thread e o ciclo de vida de uma thread. A principal conclusão prática é que, embora o modelo clássico de uma thread por tarefa seja simples de entender e programar, ele não escala bem, e é justamente esse limite que justifica o uso de ExecutorService ou, preferencialmente hoje, de Virtual Threads em aplicações que precisam lidar com um grande volume de tarefas concorrentes.